

Technical Report-Assignment 11

2371039 송승희

1. UNet Model (UNet.py)

1.1 개요

이 코드는 Pascal VOC 2012 데이터셋을 이용한 이미지 세그멘테이션을 위해 **U-Net** 아키텍처를 PyTorch로 구현한 코드이다. U-Net은 Encoder(수축 경로)와 Decoder(확장 경로)로 구성된 Encoder-Decoder 구조이며, Skip Connection을 통해 다중 스케일 정보를 동시에 활용하는 것이 핵심 특징이다.

1.2 Code Explanation

Question 1 : Implement the UNet model code.

1.2.1 conv() 함수 - 이중 컨볼루션 블록

```
def conv(in_channels, out_channels):  
    return nn.Sequential(  
        nn.Conv2d(in_channels, out_channels, 3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_channels, out_channels, 3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True)  
    )
```

U-Net의 각 단계에서 반복적으로 사용되는 기본 컨볼루션 블록이다. 3×3 Conv → BatchNorm → ReLU 구조를 두 번 연속 적용한다. padding=1 을 사용하여 입력과 출력의 공간적 해상도를 동일하게 유지한다.

파라미터	값	의미
kernel size	3	3×3 컨볼루션 필터
padding	1	공간 해상도 유지
BatchNorm2d	out_channels	학습 안정화 및 정규화
ReLU	inplace=True	비선형 활성화, 메모리 효율화

1.2.2 Unet 클래스 - __init__() : Encoder (Contracting Path)

```
self.convDown1 = conv(in_channels, 64)  
self.convDown2 = conv(64, 128)  
self.convDown3 = conv(128, 256)  
self.convDown4 = conv(256, 512)  
self.convDown5 = conv(512, 1024)
```

```
self.maxpool = nn.MaxPool2d(2, stride=2)
```

Encoder는 다섯 단계의 이중 컨볼루션 블록으로 구성된다. 각 단계 사이에 `MaxPool2d(2, stride=2)`를 적용하여 공간 해상도를 절반으로 줄이고, 채널 수는 $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$ 로 두 배씩 증가시킨다. `convDown5`는 가장 낮은 해상도에서의 bottleneck 역할을 한다.

1.2.3 Unet 클래스 - `__init__()` : Decoder (Expanding Path)

```
self.upsample = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)

self.convUp4 = conv(1536, 512) # 1024 + 512 = 1536
self.convUp3 = conv(768, 256) # 512 + 256 = 768
self.convUp2 = conv(384, 128) # 256 + 128 = 384
self.convUp1 = conv(192, 64) # 128 + 64 = 192

self.convUp_fin = nn.Conv2d(64, out_channels, 1)
```

Decoder는 `Upsample`로 해상도를 2배 복원한 후, 대응하는 Encoder 단계의 feature map을 채널 방향으로 concatenate하여 이중 컨볼루션 블록에 입력한다. Skip Connection으로 인해 각 `convUp`의 입력 채널은 upsample된 채널과 Encoder feature map 채널의 합이 된다. 최종 출력은 1×1 컨볼루션(`convUp_fin`)으로 클래스 수(`out_channels`)에 맞게 채널을 변환한다.

레이어	입력 채널 구성	출력 채널
convUp4	1024 (upsample) + 512 (convDown4)	512
convUp3	512 (upsample) + 256 (convDown3)	256
convUp2	256 (upsample) + 128 (convDown2)	128
convUp1	128 (upsample) + 64 (convDown1)	64
convUp_fin	64	out_channels

1.2.4 `forward()` 함수 - 순전파

```
def forward(self, x):
    conv1 = self.convDown1(x)
    x = self.maxpool(conv1)
    conv2 = self.convDown2(x)
    x = self.maxpool(conv2)
    ...
    conv5 = self.convDown5(x)
    x = self.upsample(conv5)

    x = torch.cat([x, conv4], dim=1)
    x = self.convUp4(x)
    x = self.upsample(x)
    ...
```

```
out = self.convUp_fin(x)
return out
```

순전파는 크게 두 단계로 나뉜다. **Encoder 단계**에서는 각 `convDown`의 출력을 skip connection용으로 저장 (`conv1 ~ conv4`)하면서 MaxPool로 해상도를 줄여나간다. **Decoder 단계**에서는 Upsample 후 `torch.cat([x, convN], dim=1)`로 채널 방향 concatenation을 수행하고, 이중 컨볼루션 블록을 통과시키는 과정을 반복한다.

2. ResNet-Encoder UNet (`resnet_encoder_unet.py`)

2.1 개요

이 코드는 기존 U-Net의 Encoder를 **ResNet-50** 기반으로 교체한 **ResNet-encoder-UNet** 아키텍처를 PyTorch로 구현한 코드이다. ResNet-50의 Layer 1~3을 Encoder로 사용하고, ImageNet으로 사전 학습된 파라미터로 초기화함으로써 더 강력한 특징 추출이 가능하다. Decoder는 기존 U-Net과 동일하게 Skip Connection을 활용한 확장 경로로 구성된다.

2.2 Code Explanation

2.2.1 `conv1x1()` / `conv3x3()` 함수 - 기본 컨볼루션 블록

```
def conv1x1(in_channels, out_channels, stride, padding):
    model = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride,
padding=padding),
        nn.BatchNorm2d(out_channels)
    )
    return model

def conv3x3(in_channels, out_channels, stride, padding):
    model = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
padding=padding),
        nn.BatchNorm2d(out_channels)
    )
    return model
```

ResNet의 Bottleneck 블록을 구성하기 위한 기본 단위 함수이다. 두 함수 모두 Conv2d와 BatchNorm2d를 묶어 반환하며, `conv1x1`은 채널 수 변환에, `conv3x3`은 공간적 특징 추출에 사용된다.

2.2.2 `ResidualBlock` 클래스 - Bottleneck 잔차 블록

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, middle_channels, out_channels, downsample=False):
```

ResNet-50의 핵심 구성 요소인 Bottleneck 블록이다. `downsample` 파라미터에 따라 두 가지 경로로 분기된다.

`downsample=True` 인 경우 (해상도 축소 + 채널 변환):

```

self.layer = nn.Sequential(
    conv1x1(in_channels, middle_channels, stride=2, padding=0),
    conv3x3(middle_channels, middle_channels, stride=1, padding=1),
    conv1x1(middle_channels, out_channels, stride=1, padding=0)
)
self.downsize = conv1x1(in_channels, out_channels, 2, 0)

```

첫 번째 conv1x1 에서 stride=2 로 해상도를 절반으로 줄이고, shortcut 경로도 downsize 로 동일하게 맞춰 residual 합산이 가능하도록 한다.

`downsample=False` 인 경우 (해상도 유지):

```

self.layer = nn.Sequential(
    conv1x1(in_channels, middle_channels, stride=1, padding=0),
    conv3x3(middle_channels, middle_channels, stride=1, padding=1),
    conv1x1(middle_channels, out_channels, stride=1, padding=0)
)
self.make_equal_channel = conv1x1(in_channels, out_channels, 1, 0)

```

모든 stride를 1로 유지하여 해상도를 보존하며, shortcut 경로는 make_equal_channel 로 채널 수만 맞춘다.

파라미터	downsample=True	downsample=False
첫 conv1x1 stride	2 (해상도 축소)	1 (해상도 유지)
shortcut 처리	downsize (stride=2)	make_equal_channel (stride=1)

2.2.3 UNetWithResnet50Encoder 클래스 - `__init__()` : Encoder (ResNet-50 기반)

```

self.layer1 = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True)
)
self.pool = nn.MaxPool2d(3, 2, 1, return_indices=True)

self.layer2 = nn.Sequential(
    ResidualBlock(64, 64, 256, False),
    ResidualBlock(256, 64, 256, False),
    ResidualBlock(256, 64, 256, True)
)
self.layer3 = nn.Sequential(
    ResidualBlock(256, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, False)
)

```

ResNet-50의 구조를 그대로 따른다. layer1 은 7x7 Conv(stride=2)와 MaxPool로 입력 해상도를 1/4로 줄이며, layer2 는 256채널 Bottleneck 블록 3개(마지막만 downsample), layer3 는 512채널 블록 4개로 구성된다.

`pool` 은 `return_indices=True` 로 `MaxUnpool`에서 위치 복원에 사용할 인덱스를 저장한다.

2.2.4 `UNetWithResnet50Encoder` 클래스 - `__init__()` : Decoder (U-Net 확장 경로)

```
self.bridge = conv(512, 512)
self.UnetConv1 = conv(512, 256)
self.UpConv1 = nn.Conv2d(512, 256, 3, padding=1)

self.upconv2_1 = nn.ConvTranspose2d(256, 256, 3, 2, 1)
self.upconv2_2 = nn.Conv2d(256, 64, 3, padding=1)

self.unpool = nn.MaxUnpool2d(3, 2, 1)
self.UnetConv2_1 = nn.ConvTranspose2d(64, 64, 3, 2, 1)
self.UnetConv2_2 = nn.ConvTranspose2d(128, 128, 3, 2, 1)
self.UnetConv2_3 = nn.Conv2d(128, 64, 3, padding=1)

self.UnetConv3 = nn.Conv2d(64, self.n_classes, kernel_size=1, stride=1)
```

`bridge` 는 Encoder 최하단에서 Decoder로 넘어가는 연결 역할을 하며, 이후 Upsample과 Skip Connection concatenation을 반복하여 해상도를 복원한다. `ConvTranspose2d` 로 학습 가능한 업샘플링을 수행하고, 최종 `UnetConv3` 의 1x1 Conv로 클래스 수 (`n_classes=22`)에 맞게 출력한다.

Question 2 : Implement the forward function of Resnet_encoder_UNet.

2.2.5 `forward()` 함수 - 순전파

```
def forward(self, x):
    out1 = self.layer1(x)
    out1, indices = self.pool(out1)
    out2 = self.layer2(out1)
    out3 = self.layer3(out2)

    x = self.bridge(out3)
    x = self.UpConv1(x)
    x = torch.cat([x, out2], dim=1) # Skip Connection 1
    x = self.UnetConv1(x)

    x = self.upconv2_1(x, output_size=torch.Size([x.size(0), 256, 64, 6
4]))
    x = self.upconv2_2(x)
    x = torch.cat([x, out1], dim=1) # Skip Connection 2
    x = self.UnetConv2_2(x, output_size=torch.Size([x.size(0), 128, 128, 1
28]))
    x = self.UnetConv2_2(x, output_size=torch.Size([x.size(0), 128, 256, 2
56]))
    x = self.UnetConv2_3(x)
    x = self.UnetConv3(x)
    return x
```

Encoder에서 `out1` (layer1 출력), `out2` (layer2 출력), `out3` (layer3 출력)을 각각 저장하여 Skip Connection에 활용한다. Decoder에서는 `torch.cat([x, out2], dim=1)` 과 `torch.cat([x, out1], dim=1)` 으로 대응하는 Encoder feature map을 채널 방향으로 연결하고, `ConvTranspose2d` 의 `output_size` 인자로 업샘플링 후의 정확한 해상도를 명시하여 크기 불일치를 방지한다.

Skip Connection	Encoder 출력	연결 위치
1번째	out2 (layer2, 256ch)	bridge → UnetConv1 사이
2번째	out1 (layer1, 64ch)	upconv2_2 → UnetConv2_2 사이

3. Train/Test Module (`modules_skeleton.py`)

3.1 개요

이 코드는 U-Net 및 ResNet-encoder-UNet 모델의 학습(train), 손실 및 정확도 평가(get_loss_train), 검증 및 시각화(val_model) 기능을 담당하는 모듈이다. Pascal VOC 2012의 21개 클래스(+background)에 대해 픽셀 단위 세그멘테이션을 수행하며, 예측 결과를 클래스별 RGB 색상으로 변환하여 이미지로 저장한다.

3.2 Code Explanation

Question 3: Implement the train/test module.

3.2.1 `train_model()` 함수 - 모델 학습

```
def train_model(trainloader, model, criterion, optimizer, scheduler, device):
    model.train()
    for i, (inputs, labels) in enumerate(trainloader):
        inputs = inputs.to(device)
        labels = labels.to(device=device, dtype=torch.int64)
        criterion = criterion.cuda()

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        scheduler.step()
```

학습 루프의 핵심 단계는 다음 순서로 진행된다.

단계	코드	역할
1. 기울기 초기화	<code>optimizer.zero_grad()</code>	이전 배치의 기울기 누적 방지
2. 순전파	<code>outputs = model(inputs)</code>	모델 예측값 계산
3. 손실 계산	<code>loss = criterion(outputs, labels)</code>	Cross Entropy Loss 계산
4. 역전파	<code>loss.backward()</code>	기울기 계산
5. 파라미터 갱신	<code>optimizer.step()</code>	Adam optimizer로 가중치 업데이트

단계	코드	역할
6. 스케줄러 갱신	<code>scheduler.step()</code>	학습률 조정

`labels` 는 픽셀 단위의 클래스 인덱스이므로 `dtype=torch.int64` 로 변환하여 Cross Entropy Loss에 입력한다.

3.2.2 `accuracy_check()` / `accuracy_check_for_batch()` 함수 - 정확도 계산

```
def accuracy_check(label, pred):
    compare = np.equal(np.array(label), np.array(pred))
    accuracy = np.sum(compare)
    return accuracy / len(np.array(label).flatten())

def accuracy_check_for_batch(labels, preds, batch_size):
    total_acc = 0
    for i in range(batch_size):
        total_acc += accuracy_check(labels[i], preds[i])
    return total_acc / batch_size
```

픽셀 단위 정확도를 계산하는 함수이다. `accuracy_check()` 는 정답 레이블과 예측값을 numpy 배열로 변환한 뒤 `np.equal()` 로 픽셀별 일치 여부를 비교하고, 전체 픽셀 수 대비 일치 픽셀 수의 비율을 반환한다. `accuracy_check_for_batch()` 는 배치 내 각 샘플의 정확도를 평균하여 배치 단위 정확도를 산출한다.

3.2.3 `get_loss_train()` 함수 - 학습 데이터 손실 및 정확도 평가

```
def get_loss_train(model, trainloader, criterion, device):
    model.eval()
    for batch, (inputs, labels) in enumerate(trainloader):
        with torch.no_grad():
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            outputs = np.transpose(outputs.cpu(), (0, 2, 3, 1))
            preds = torch.argmax(outputs, dim=3).float()
            acc = accuracy_check_for_batch(labels.cpu(), preds.cpu(), inputs.size()[0])
```

`model.eval()` 과 `torch.no_grad()` 로 기울기 계산을 비활성화하여 메모리를 절약하고 평가를 수행한다. 모델 출력인 `outputs` 는 `(N, C, H, W)` 형태이므로 `np.transpose` 로 `(N, H, W, C)` 로 변환한 뒤, `torch.argmax(dim=3)` 으로 각 픽셀의 최대 확률 클래스 인덱스를 예측값으로 취한다.

3.2.4 `val_model()` 함수 - 검증 및 세그멘테이션 결과 시각화

```
cls_invert = {0: (0, 0, 0), 1: (128, 0, 0), 2: (0, 128, 0), ...
              21: (224, 224, 192)}
```

Pascal VOC 2012의 22개 클래스(background 포함)를 각각 고유한 RGB 색상에 매핑한 딕셔너리이다. 검증 루프에서 예측값과 정답 레이블 모두 이 딕셔너리를 통해 RGB 이미지로 변환한다.

```

for j in range(temp.shape[0]):
    for k in range(temp.shape[1]):
        temp_label[j, k] = cls_invert[int(temp_l[j, k])] # 정답 마스크
        temp_rgb[j, k] = cls_invert[int(temp[j, k])] # 예측 마스크

img_print.save(dir + str(n) + 'label' + '.png')
mask_print.save(dir + str(n) + 'result' + '.png')

```

픽셀 단위로 이중 루프를 순회하며 각 픽셀의 클래스 인덱스를 `cls_invert` 에서 RGB 튜플로 변환한다. 정답 마스크는 `*label.png` 로, 예측 마스크는 `*result.png` 로 저장하여 육안으로 세그멘테이션 결과를 비교할 수 있도록 한다.

저장 파일	내용
<code>n_label.png</code>	정답 세그멘테이션 마스크 (RGB 변환)
<code>n_result.png</code>	모델 예측 세그멘테이션 마스크 (RGB 변환)

4. Main Code (`main_skeleton.py`)

4.1 개요

이 코드는 Pascal VOC 2012 데이터셋을 불러오고, 모델 초기화·학습·검증·저장까지 전체 파이프라인을 실행하는 메인 코드이다. 사전 학습된 체크포인트(`resnet_encoder_unet.pth`)가 존재하면 이를 불러와 1 epoch만 학습하고, 없으면 처음부터 40 epoch 학습한다. 매 4 epoch마다 모델 파라미터를 저장하며, 학습 종료 후 Loss/Accuracy 히스토리를 그래프로 저장한다.

4.2 Code Explanation

4.2.1 데이터셋 로드 및 전처리

```

transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize([resize_size, resize_size], PIL.NEAREST),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

trainset = Loader(data_dir, flag='train', resize=resize_size, transforms=transforms)
valset = Loader(data_dir, flag='val', resize=resize_size, transforms=transforms)

trainLoader = DataLoader(trainset, batch_size=batch_size, shuffle=True)
validLoader = DataLoader(valset, batch_size=batch_size, shuffle=True)

```

`datasets.py`에서는 데이터셋을 불러오고 사용하기 위한 Dataset 클래스를 구현한다. `check_exists_()` 함수에서 데이터셋 저장경로가 유효한지 확인하고 `VOCdataloader()`에서 이미지와 label 파일을 전처리해 반환하며, `len()` 함수와 `getitem()` 함수에서는 각각 데이터셋의 이미지 개수를 반환하고 이미지와 마스크 튜플을 반환한다 `init()`에

서는 지정된 경로로부터 데이터를 읽어 그 중 80%를 훈련용 데이터셋으로, 나머지 20%를 테스트용 데이터셋으로 분류한다.

Pascal VOC 2012 데이터를 256×256으로 리사이즈한 뒤 텐서로 변환하고, ImageNet 통계값으로 정규화한다. 리사이즈 시 `PIL.NEAREST` 보간법을 사용하여 세그멘테이션 레이블의 클래스 인덱스가 왜곡되지 않도록 한다.

전처리 단계	내용
Resize	256×256, NEAREST 보간
Normalize mean	(0.485, 0.456, 0.406)
Normalize std	(0.229, 0.224, 0.225)

4.2.2 모델 초기화 및 손실함수 / 옵티마이저 설정

```
model = UNetWithResnet50Encoder(n_classes=22)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = StepLR(optimizer, step_size=4, gamma=0.1)
```

Pascal VOC 2012의 클래스 수(background 포함 22개)에 맞춰 모델을 초기화한다. 손실함수는 픽셀 단위 다중 클래스 분류에 적합한 `CrossEntropyLoss` 를 사용하고, 옵티마이저는 `Adam` (lr=0.001)을 적용한다. `StepLR` 스케줄러는 매 4 step마다 학습률을 0.1배로 감소시켜 학습 후반부의 수렴을 돕는다.

설정 항목	값
모델	UNetWithResnet50Encoder
클래스 수	22 (background 포함)
손실함수	CrossEntropyLoss
옵티마이저	Adam (lr=0.001)
스케줄러	StepLR (step=4, gamma=0.1)

4.2.3 사전 학습 모델 로드

```
pretrained_path = "resnet_encoder_unet.pth"

try:
    model.load_state_dict(torch.load(pretrained_path, map_location=device))
    print("pretrained checkpoint loaded")
    epochs = 1
except FileNotFoundError:
    print("no file")
```

`torch.load()` 로 저장된 파라미터를 불러오고 `load_state_dict()` 로 모델에 적용한다. `map_location=device` 를 지정하여 GPU/CPU 환경에 무관하게 로드가 가능하다. 체크포인트가 존재하면 `epochs=1` 로 설정하여 추가 학습을 최소화하고, 파일이 없으면 40 epoch 전체를 학습한다.

4.2.4 학습 루프 및 모델 저장

```

for epoch in range(epochs):
    train_model(trainLoader, model, criterion, optimizer, scheduler, device)
    train_acc, train_loss = get_loss_train(model, trainLoader, criterion, device)
    val_acc, val_loss = val_model(model, validLoader, criterion, device, predict_save_folder)

    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    if epoch % 4 == 0:
        torch.save(model.state_dict(), savepath2)

```

매 epoch마다 `train_model()` 로 학습 후 `get_loss_train()` 과 `val_model()` 로 학습/검증 손실 및 정확도를 측정한다. 결과는 `history` 딕셔너리에 누적하며, 4 epoch마다 `torch.save(model.state_dict(), path)` 로 모델 파라미터를 `.pth` 파일로 저장한다.

4.2.5 학습 결과 시각화

```

plt.subplot(2,1,1)
plt.plot(range(epoch+1), history['train_loss'], color='red')
plt.plot(range(epoch+1), history['val_loss'], color='blue')

plt.subplot(2,1,2)
plt.plot(range(epoch+1), history['train_acc'], color='red')
plt.plot(range(epoch+1), history['val_acc'], color='blue')

plt.savefig(result_save_dir + 'result')

```

학습 완료 후 Loss와 Accuracy의 epoch별 변화를 2개의 서브플롯으로 시각화하여 저장한다. 빨간색은 학습 데이터, 파란색은 검증 데이터를 나타내며, 두 곡선의 간격을 통해 과적합 여부를 판단할 수 있다.

5. 결과 분석

Google Colab Pro를 사용하고 있어서 로컬 대신에 코랩환경에서 위 코드들과 T4 GPU를 이용해 학습한 결과는 아래와 같다.

5.1 UNet 결과 분석

```
Loading datasets...
Train: 2331장, Val: 582장
Device: cuda:0
pretrained checkpoint loaded!
Training...
[Train] Iter 0/583, loss = 1.4268
[Train] Iter 10/583, loss = 1.1794
[Train] Iter 20/583, loss = 0.6810
[Train] Iter 30/583, loss = 2.6994
[Train] Iter 40/583, loss = 3.6006
[Train] Iter 50/583, loss = 1.5799
[Train] Iter 60/583, loss = 1.2195
[Train] Iter 70/583, loss = 1.1213
[Train] Iter 80/583, loss = 1.4102
[Train] Iter 90/583, loss = 1.2614
[Train] Iter 100/583, loss = 1.5052
[Train] Iter 110/583, loss = 1.3963
[Train] Iter 120/583, loss = 1.4107
[Train] Iter 130/583, loss = 2.0461
[Train] Iter 140/583, loss = 1.0887
[Train] Iter 150/583, loss = 1.4848
[Train] Iter 160/583, loss = 1.0636
[Train] Iter 170/583, loss = 1.6053
[Train] Iter 180/583, loss = 6.0094
[Train] Iter 190/583, loss = 0.6326
[Train] Iter 200/583, loss = 1.9584
[Train] Iter 210/583, loss = 2.1119
[Train] Iter 220/583, loss = 0.8243
[Train] Iter 230/583, loss = 1.5172
[Train] Iter 240/583, loss = 0.8490
[Train] Iter 250/583, loss = 2.2628
[Train] Iter 260/583, loss = 0.9446
[Train] Iter 270/583, loss = 0.5687
[Train] Iter 280/583, loss = 0.8748
[Train] Iter 290/583, loss = 1.5064
[Train] Iter 300/583, loss = 1.4899
[Train] Iter 310/583, loss = 0.8629
[Train] Iter 320/583, loss = 1.0484
[Train] Iter 330/583, loss = 0.9069
```

```
[Train] Iter 340/583, loss = 0.8597
[Train] Iter 350/583, loss = 0.8551
[Train] Iter 360/583, loss = 2.5347
[Train] Iter 370/583, loss = 0.9335
[Train] Iter 380/583, loss = 0.8339
[Train] Iter 390/583, loss = 1.3173
[Train] Iter 400/583, loss = 1.8221
[Train] Iter 410/583, loss = 1.6873
[Train] Iter 420/583, loss = 0.9167
[Train] Iter 430/583, loss = 0.7657
[Train] Iter 440/583, loss = 1.4869
[Train] Iter 450/583, loss = 1.2172
[Train] Iter 460/583, loss = 1.0037
[Train] Iter 470/583, loss = 0.7163
[Train] Iter 480/583, loss = 0.8583
[Train] Iter 490/583, loss = 1.0925
[Train] Iter 500/583, loss = 0.8697
[Train] Iter 510/583, loss = 1.0927
[Train] Iter 520/583, loss = 1.3022
[Train] Iter 530/583, loss = 1.7222
[Train] Iter 540/583, loss = 0.8442
[Train] Iter 550/583, loss = 1.6583
[Train] Iter 560/583, loss = 1.5690
[Train] Iter 570/583, loss = 1.3798
[Train] Iter 580/583, loss = 1.1954
epoch 1 train loss: 1.0952, train acc: 0.8185
epoch 1 val loss: 1.5280, val acc: 0.7318
Finish Training!
```

UNet 모델을 Pascal VOC 2012 데이터셋으로 1 epoch 학습한 결과, train loss 1.0952, train accuracy 0.8185, val loss 1.5280, val accuracy 0.7318을 달성하였다. 사전 학습된 체크포인트 (UNet_trained_model.pth)를 로드한 후 1 epoch 추가 학습을 진행하였으며, train accuracy와 val accuracy 간의 차이(0.8185 vs 0.7318)는 모델이 학습 데이터에 다소 과적합되어 있음을 나타낸다. 그러나 val accuracy 0.7318은 비교적 높은 수치로, 사전 학습된 가중치가 Pascal VOC 2012의 세그멘테이션 태스크에 효과적으로 적용되었음을 보여준다.

5.2 ResNet-encoder-UNet 결과 분석

```
Loading datasets...
Train: 2331장, Val: 582장
Device: cuda:0
pretrained checkpoint loaded!
Training ResNet-encoder-UNet...
[Train] Iter 0/583, loss = 2.6545
[Train] Iter 10/583, loss = 1.8790
[Train] Iter 20/583, loss = 1.2664
[Train] Iter 30/583, loss = 1.7705
[Train] Iter 40/583, loss = 2.4083
[Train] Iter 50/583, loss = 0.9307
[Train] Iter 60/583, loss = 1.5875
[Train] Iter 70/583, loss = 2.3055
[Train] Iter 80/583, loss = 2.1051
[Train] Iter 90/583, loss = 2.0751
[Train] Iter 100/583, loss = 1.2491
[Train] Iter 110/583, loss = 1.1841
[Train] Iter 120/583, loss = 1.1666
[Train] Iter 130/583, loss = 0.9127
[Train] Iter 140/583, loss = 1.7700
[Train] Iter 150/583, loss = 1.7218
[Train] Iter 160/583, loss = 1.2189
[Train] Iter 170/583, loss = 2.3536
[Train] Iter 180/583, loss = 1.3376
[Train] Iter 190/583, loss = 3.7380
[Train] Iter 200/583, loss = 1.0734
[Train] Iter 210/583, loss = 2.5160
[Train] Iter 220/583, loss = 2.4734
[Train] Iter 230/583, loss = 1.9106
[Train] Iter 240/583, loss = 2.2949
[Train] Iter 250/583, loss = 1.9903
[Train] Iter 260/583, loss = 1.5469
[Train] Iter 270/583, loss = 1.7379
[Train] Iter 280/583, loss = 1.4914
[Train] Iter 290/583, loss = 1.5215
[Train] Iter 300/583, loss = 1.3963
[Train] Iter 310/583, loss = 1.9215
[Train] Iter 320/583, loss = 1.7703
[Train] Iter 330/583, loss = 1.4301
[Train] Iter 340/583, loss = 2.6654
[Train] Iter 350/583, loss = 1.0824
[Train] Iter 360/583, loss = 2.0499
[Train] Iter 370/583, loss = 3.7901
[Train] Iter 380/583, loss = 2.3722
[Train] Iter 390/583, loss = 0.9477
[Train] Iter 400/583, loss = 1.7425
[Train] Iter 410/583, loss = 2.4397
[Train] Iter 420/583, loss = 2.3866
```

```
[Train] Iter 420/583, loss = 2.3866
[Train] Iter 430/583, loss = 1.6318
[Train] Iter 440/583, loss = 1.7845
[Train] Iter 450/583, loss = 1.3075
[Train] Iter 460/583, loss = 1.6667
[Train] Iter 470/583, loss = 2.3544
[Train] Iter 480/583, loss = 1.4750
[Train] Iter 490/583, loss = 1.4030
[Train] Iter 500/583, loss = 2.7050
[Train] Iter 510/583, loss = 1.0750
[Train] Iter 520/583, loss = 1.4468
[Train] Iter 530/583, loss = 1.3822
[Train] Iter 540/583, loss = 2.3074
[Train] Iter 550/583, loss = 1.5875
[Train] Iter 560/583, loss = 1.9172
[Train] Iter 570/583, loss = 2.1642
[Train] Iter 580/583, loss = 1.3061
epoch 1 train loss: 1.8816, train acc: 0.6550
epoch 1 val loss: 1.9354, val acc: 0.6443
Finish Training!
```

ResNet-50을 인코더로 사용한 UNet 모델의 경우, 1 epoch 학습 결과 train loss 1.8816, train accuracy 0.6550, val loss 1.9354, val accuracy 0.6443을 기록하였다. Original UNet과 비교하여 train accuracy가 0.8185에서 0.6550으로, val accuracy가 0.7318에서 0.6443으로 낮게 나타났다. 이는 ResNet 인코더를 결합한 구조가 더 복잡한 만큼 충분한 학습이 이루어지지 않았기 때문으로 추측된다. 또한 val loss(1.9354)가 train loss(1.8816)와 유사한 수준으로, Original UNet에 비해 과적합이 적게 발생하였음을 확인할 수 있다.